AEGIS TECHNICAL DOCUMENTATION

Architecture and implementation reference

28 October 2026 · Companion to AEGIS_USER_GUIDE

This document explains the AEGIS system that exists, the product we intend to build, and the work needed to connect the two. It is written for engineers and coding agents who need enough detail to implement a component without inventing contracts or mistaking a proposal for working software.

AEGIS aims to protect software across execution environments by combining compatible protection backends, functional verification, provenance, and controlled reverse-engineering evaluation. The current implementation is a local orchestration platform with one real backend: a prototype LLVM transformation.

Read the part you need

Section	Use it for
01 Project and scope	Product intent, threat model, and the meaning of protection.
02 Architecture	Component boundaries, execution, storage, and limitations.
03 Core contracts	Models, lifecycle rules, API, and CLI behavior.
04 Adapter protocol	The actual subprocess interface and failure rules.
05 LLVM Shield	Selection, transformations, semantics, and current limits.
06 ImageLab	The reference application and recorded integration evidence.
07 Platform support	Native, Rust, web, Go, game, and device integration.
08 Evaluation	Fair reconstruction trials and generalization checks.
09 Development runbook	Setup, real adapter tests, and the application workflow.
10 Team implementation briefs	The four roles, dependencies, and acceptance criteria.
11 Integration and release	CI, merge requirements, rollout order, and open decisions.
12 Agent working brief	A complete assignment prompt and handoff format.
13 Reference index	Source links, authority, and terms used in both documents.

Baseline and authority

Repository: Youssef-Boussabah/AEGIS. Reviewed baseline: **c99585795eff3708823ed620a3b37496f7a84a30** on `dev/integration`. Public repository metadata, source files, and issues were inspected for this revision. Build and test results below are attributed records; no compiler or application tests were rerun while preparing these documents.

Implemented means present in the reviewed source. **Recorded** identifies a result from a commit or earlier project run. **Proposed** identifies the design or task to build next. Repository rules govern edits; source and tests determine actual behavior. Resolve a conflict instead of silently treating this PDF as a new interface specification.

01 Project and scope

Product intent

AEGIS should help software owners protect valuable implementation details while retaining control of correctness, performance, and deployment. A user should be able to select a protection profile, run it against a supported build, inspect what changed, and compare the outcome with an unprotected build under declared attack conditions.

The ambition includes desktop applications, browser-delivered software, mobile apps, services, games, embedded software, and console software built through authorized development paths. Support belongs to a tested combination of language, compiler, artifact format, operating system, and build settings. A single successful C++ experiment does not imply support for other combinations.

What we are trying to protect

The initial research target is recovery of selected program behavior from a shipped artifact. An analyst may inspect instructions, use a decompiler, execute a local copy, observe inputs and outputs, and ask an AI model to reconstruct a function. A trial must state which of those capabilities are available.

The owner retains source code and private build information. Evaluation workers receive only the material permitted by the trial. Hidden tests and signing keys remain outside the worker's access. Public source availability creates a separate contamination risk; Section 08 describes how to account for it.

Distinct claims require distinct evidence

Claim	Evidence needed
Behavior is preserved	Independent functional checks over a stated input domain.
Transformation survives the build	Inspection of the final linked artifact under the actual optimization settings.
Reconstruction is harder	Paired attacks with equivalent access and budgets, scored independently.
A build record is authentic	Signature verification against an explicitly trusted public key.
A watermark is recoverable	A detector, false-positive controls, and tests after defined transformations.
A policy is enforceable	A cooperating enforcement mechanism; publishing metadata alone is insufficient.

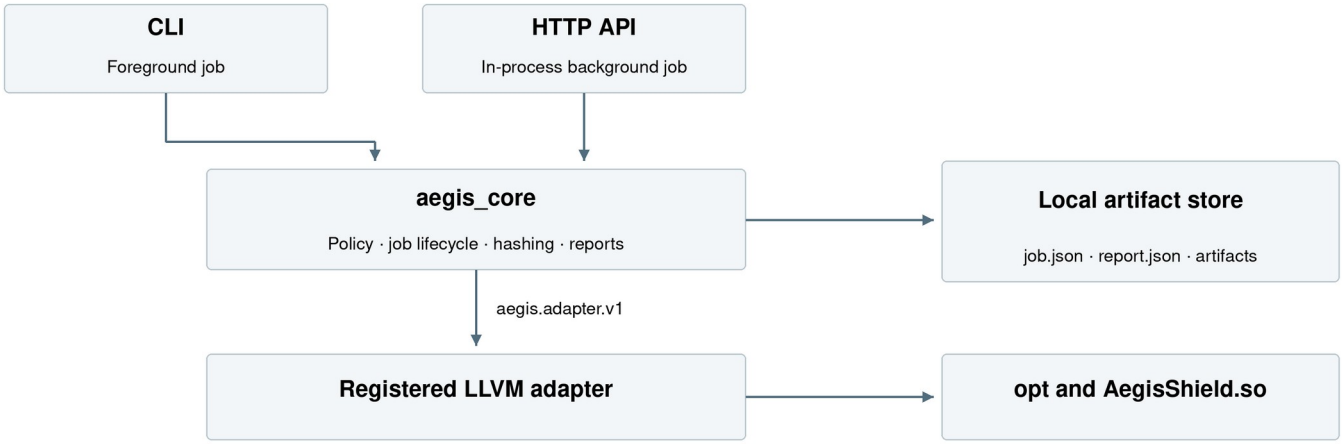
Shield addresses program structure. Mirage explores decoys. Signature covers signed provenance and separately researched watermarking. Sentinel carries rights and usage policy for cooperating tools. The Lab evaluates attacks. The platform coordinates jobs and will later select candidates using measured trade-offs.

Browser code and assets delivered to a client remain observable. Client-side protection must not become a place to store server secrets. Native execution on an analyst-controlled machine also exposes behavior. Our goal is a measured increase in effort, not an absolute promise that software cannot be understood.

Platform signing, authorization, and distribution requirements remain part of each integration. Protecting source or a compiled routine does not by itself protect an entire game's assets, a hosted service, or a console platform.

02 Architecture

The implemented system



The CLI and FastAPI service both use `aegis_core`. Core owns validation, storage, the job lifecycle, adapter execution, and reports. It depends on Pydantic v2 and the standard library; it does not import the API, CLI, or a protector. The API and CLI depend on Core, not on one another. [S1, S2]

A backend is a separately registered executable. Core invokes it through `aegis.adapter.v1`. The LLVM adapter runs `opt` with `AegisShield.so`; neither the plugin nor its compiler-specific semantics are imported into Core. This boundary is how additional backends can join the same platform.

Job execution and persistence

Submission validates the policy, creates the job directories, copies and hashes the input, and writes a pending job. Running claims the job atomically, moves it to running, rechecks the policy, invokes the adapter, and validates the result. Core writes the report before the terminal job record. [S2]

Files live under `<store>/jobs/<job_id>/`: `inputs/`, `outputs/`, `adapter/`, and `work/`, plus `job.json` and `report.json`. The default store is `.aegis/` beside the configuration. JSON writes use temporary files and atomic replacement. There is no database, durable queue, or remote artifact service in this design.

The CLI runs a job in the foreground. The API starts it as an in-process background task. If the API process stops during execution, a job can remain marked running; automatic recovery is not implemented. The generic adapter runner buffers stdout and stderr in memory and checks its stdout limit after the process exits. [S2]

Boundaries future work must preserve

Adapters use argument arrays, a minimal environment, job-local paths, and a wall-clock timeout that kills the process group. These controls do not form a security sandbox. Running untrusted reconstructed programs requires a separately designed isolation layer.

The current job accepts one input artifact and one backend. Chaining protection, compilation, tests, signatures, and Lab trials is proposed product integration. Keep intermediate artifacts and parent-child relationships explicit when that workflow is designed; do not silently redefine the existing job contract.

03 Core contracts

Shared data model

Definitions live in `core/src/aegis_core/models.py`. Unknown fields are rejected. Core's `schema_version` is string `"1"`; Shield's local schemas use integer `1`. These are separate contracts. [S3]

Model	Meaning and important constraints
ProtectionPolicy	Backend, task, scalar options, optional timeout and authorization. At most 64 options; string values at most 4096 characters; floats must be finite.
ProtectionJob	Input and output artifacts, policy, status, timestamps, and an optional structured error.
Artifact	A safe name, role, size, SHA-256, and normalized store-relative path. Core computes hashes.
EvidenceRecord	Evidence kind, summary/data, and source: orchestrator or adapter.
AttackEvaluation	Attack outcome tied to an artifact hash. Outcomes are resisted, bypassed, inconclusive, or error.
ProtectionReport	Terminal job summary, evidence, evaluations, errors, and explicit limitations.
AttackLabAuthorization	Nonblank authorized_by, reference, and scope; attestation time and optional expiry.

Job status moves from `pending` to `running` to `succeeded` or `failed`. A failed job carries an error; a successful one does not. Terminal jobs have `finished_at`. Only successful jobs list output artifacts. Reports represent terminal jobs. Rejected submissions can fail before any job exists.

Evidence marked `orchestrator` describes what Core observed or verified, such as artifact hashes and invocation results. Transformation counts and attack results returned by a backend remain `adapter` claims. Recording a claim does not independently validate it.

API and CLI surface

Operation	Implemented interface
Health	GET /healthz; CLI aegis inspect
Submit	POST /v1/jobs with policy and base64 input
Status	GET /v1/jobs/{job_id}; CLI aegis inspect JOB_ID
Report	GET /v1/jobs/{job_id}/report; CLI aegis report JOB_ID
Protect locally	aegis protect FILE --backend NAME

The submit endpoint returns 202 and a pending job. A report requested too early returns `REPORT_NOT_READY`. The present API has no job-list or artifact-download endpoint; the dashboard must coordinate those additions with integration. Core's internal listing method does not create an HTTP route. [S4]

The API defaults to `127.0.0.1`; non-loopback binding requires `AEGIS_API_TOKEN`, which enables bearer authentication on `/v1/*`. Attack tasks require the operator switch and a per-request authorization, checked at submission and execution. Task: `attack_lab`. CLI: `--task attack-lab`. [S2, S4]

04 Adapter protocol

Invocation and request

Register an adapter in `aegis.toml` under `[adapters.<name>]`. Core invokes the trusted command array followed by `--aegis-request` and an absolute request-file path. stdin is closed; cwd is the job's `work/` directory. Options and input names travel inside JSON, not an interpolated shell command. [S5]

The request fields are `protocol`, `job_id`, `task`, `backend`, `policy_name`, `input_path`, `input_sha256`, `output_dir`, `options`, and `authorization`. The protocol is `aegis.adapter.v1`. Input and output locations are absolute. Authorization is supplied for attack tasks.

The adapter must leave the input unchanged, write outputs inside the supplied directory, print one result JSON object to stdout, and send diagnostics to stderr. It receives only the minimal environment plus explicitly allowed variables. Child processes must remain subject to the supervising timeout.

Result structure

This example uses the existing wire format. Evidence entries, when present, contain a summary and data object.

```
{
  "protocol": "aegis.adapter.v1",
  "status": "succeeded",
  "outputs": [{"path": "protected.ll"}],
  "evidence": [],
  "attack_evaluations": [],
  "error": null
}
```

Output paths are relative to `output_dir`. Core validates files, recomputes their hashes, and maps evaluation targets to artifact hashes. An evaluation target can name `input`, an output path, or be null. Adapter-supplied hashes never replace Core's own calculation.

Failure is part of the contract

Condition	Core result
Missing executable or timeout	ADAPTER_UNAVAILABLE or ADAPTER_TIMEOUT
Nonzero exit or reported failure	ADAPTER_FAILED
Invalid JSON, unsafe or missing output	ADAPTER_INVALID_OUTPUT
Protect success without outputs	ADAPTER_INVALID_OUTPUT
Attack success without evaluations	ADAPTER_INVALID_OUTPUT
Attack evaluations in a protect task	ADAPTER_INVALID_OUTPUT
Input changed during execution	ARTIFACT_INTEGRITY_ERROR

An LLVM-adapter failure can exit zero with status failed. Core returns ADAPTER_FAILED and retains the adapter's error details. Exit zero alone is not success. [S5, S7]

05 LLVM Shield

Implemented transformation

Shield is a C++20 LLVM new-pass-manager plugin. It runs an explicit module pass named `aegis-shield`. Loading it with `-fpass-plugin` alone does not schedule that pass. The platform adapter supports LLVM 22.x and a plugin built against the same LLVM installation. Recorded validation used 22.1.8 on Linux x86-64. [S6, S7]

A selected function has the exact source annotation `aegis.protect` or the corresponding IR string attribute. It must be a definition with a scalar integer return value, scalar integer parameters, no variadic arguments, and no `optnone`. Pointer-parameter functions are outside this initial selection rule.

Within selected functions, the pass expands eligible scalar integer additions of widths 2 through 128. It skips vectors, overflow-flagged adds, direct undef/poison operands, and unsupported widths. Control flow, memory operations, calling conventions, signatures, and other opcodes are not transformed.

```
Arithmetic modulo 2^width

a + b = (a XOR b) + ((a AND b) << 1)
a + b = (a OR b)  +  (a AND b)
```

These identities explain equality for defined bitvector inputs. The implementation freezes operands before duplicate use. For undef or poison, this is a permitted refinement, not identical propagation of every undefined value. Signed overflow and general program undefined behavior are not made safe by the pass. [S6]

Options and reporting

Adapter option	Default	Accepted value
seed	"0"	Decimal string in the uint64 range.
max-sites	16	Integer from 0 to 1,000,000; budget per function.
enabled	true	Boolean.

Site selection is deterministic from the function name, seed, and original add ordinal. The seed is not a secret key. Generated-instruction metadata prevents immediate repeated expansion, but later passes can remove it. Reports record effective configuration, LLVM version, function reasons, and site counts.

The adapter accepts textual LLVM IR, not source files, executables, or binary bitcode. It validates options and paths, stages input privately, runs the fixed Shield pipeline, checks the report and artifacts, and performs another LLVM verification. Success requires nonzero reported sites and changed, nonempty output. Zero-site jobs fail with `NO_TRANSFORMATIONS_APPLIED`. [S7]

What remains to establish

Frontend optimization can remove candidate adds or attach flags that make them ineligible. Later optimization can simplify expansions. Correct execution at O2/O3 does not show that the intended structure survived. Shield v2 must inspect the final linked artifact and compare reconstruction outcomes; more IR instructions alone are not evidence of resistance.

06 ImageLab and recorded evidence

Why this application exists

ImageLab is our C++20 image-processing library, CLI, and measurement harness. It contains three selected kernels: tone mapping, color adjustment, and enhancement. Their integer-only C interfaces fit Shield's current selection rules. That accommodation makes ImageLab useful for controlled regression, but unrepresentative of every production program. [S8]

Its C++ tests use a separate signed-arithmetic reference, while the Python model uses exact fractional arithmetic. The suite also checks image encoding, parameter rejection, annotations, known outputs, and comparisons between builds. No single test set proves correctness for every possible application input.

Build variants

Variant	Purpose
native	The ordinary source build used to expose real release-build costs.
ir-baseline	Unprotected kernels built through the same IR route as transformed kernels.
shield	Kernels transformed directly by the plugin during the ImageLab build.
external	Kernels supplied by another workflow, including AEGIS's protected.ll.

The prepared `kernels.input.ll` comes from an O0-based frontend pipeline that manages the `optnone` issue. Do not use already transformed `kernels.shield.ll` as the baseline. The CMake external hook accepts `.ll` or `.bc`; the AEGIS adapter accepts textual `.ll` only. [S8]

Evidence register

Record	Source and result	Interpretation
Combined Python run	Commit c995857: Core 70, API 17, CLI 18, adapter 94; total 199 passed.	Recorded regression coverage; one existing Starlette/httpx warning.
Shield tests	Same commit: 1 CTest entry containing 14 cases passed.	Compiler and runtime checks, not 14 separate CTest entries.
First CLI fixture	User-supplied terminal report; job d5b8a74edb3c46ec80a3a058c6caf885; 1 site.	Directly shown protocol run and adapter report.
ImageLab application job	Earlier project handoff; job 4001e5aceb144882a21a2f6653e52fb3; 43 sites, 3 functions; 11/11 external CTests.	Recorded milestone; raw artifacts were not re-opened for this revision.

The earlier handoff gives the 43-site breakdown as 9 tone-map, 24 color-adjust, and 10 enhancement sites. Preserve the original report and executable hashes before using that result in a submission. The 11-test external configuration differs from the 12-test direct-Shield configuration, which also includes the Shield-specific check. [S8, S9]

No record above establishes AI reconstruction resistance. The documentation work did not rerun these suites. New claims should cite the new commit, commands, toolchain, and saved result files.

07 Platform support and external targets

Add support by execution environment

The same orchestration can serve different ecosystems, but each backend needs its own eligibility rules and build integration. The following routes are proposed work, except for the current textual LLVM path. A support claim should name the exact tested profile.

Environment	Candidate integration	What must be validated
Native C and C++	Compiler IR or another native build stage.	ABI, exceptions, optimization, link steps, debug information, and final behavior.
Rust	A Rust-compatible compiler or object-level route.	rustc/LLVM compatibility, monomorphization, panic behavior, signatures, and supported selection.
Go	A Go-aware build transformation or compatible native backend.	Go runtime and toolchain assumptions; no direct LLVM route is established.
Browser JavaScript and TypeScript	AST or bundler integration; possibly specific WebAssembly components.	Semantics, browser behavior, bundles, source maps, startup, and accessibility.
Mobile and games	Native, script, and asset paths assessed separately.	Frame time, packaging, signing, supported runtimes, and asset handling.
Embedded and console builds	Authorized toolchain and packaging integrations.	Device limits, architecture, platform requirements, and available test hardware.

An application may contain several of these. A protected server binary does not protect its browser bundle, and transforming game logic does not establish protection of textures or scripts.

Requested external evaluation set

PhotoCraft is a Rust photo editor. Start with an independently buildable component and its output checks. Establish a compatible transformation route before claiming support; rustc using LLVM internally is not enough. [S11]

reVC is the Vice City branch in the archived re-GTA mirror. Pin that branch's commit, document build dependencies, and establish the availability of required game data before planning a full runtime demo. Use only assets the team is authorized to use and distribute. [S12]

Mattermost separates server and web-client concerns. Evaluate the Go server and browser client as different profiles, with separate builds, tests, and protection claims. A functioning application smoke test is useful alongside targeted component tests. [S13]

These are selected targets, not completed integrations. Capture upstream revision, build commands, dependency versions, license/asset inventory, input fixtures, and a baseline before applying a protector. Keep application-specific harness code outside the engine.

Benchmark issue #7 tracks PhotoCraft, reVC and Mattermost. These remain evaluation targets, not special cases in the engine. Pin revisions and establish baseline builds before claiming support. [S10]

08 Evaluation design

Start with a question that can be answered

For a selected function and a fixed attacker budget, does the protected artifact reduce successful behavioral reconstruction relative to its unprotected counterpart? Define success before running trials. A useful first task is to implement the target function well enough to pass independent hidden tests, rather than to recover the original source text.

Source-text similarity can be reported as a diagnostic, but equivalent code can look different. A working reconstruction is the important outcome. Also distinguish a candidate that fails to compile from one that compiles and fails behavior tests.

Build the controls first

Use an ordinary optimized native build, an unprotected build through the same IR pipeline, and the protected build. The matched pipeline isolates the transformation's effect; the ordinary release build exposes costs introduced by the integration itself. Mirage experiments additionally need a no-decoy control. Keep symbols, debug information, libraries, and oracle access equivalent across paired conditions.

Run functional checks before attacks. A broken protected build is ineligible for an effectiveness claim. Inspect the linked executable or actual deployed bundle after all optimization and stripping. Verify coverage: which requested functions were eligible, transformed, retained, and tested.

Keep the comparison fair

1. Freeze the source revision, compiler settings, protection configuration, attack prompt, tool versions, model identifier, and success criterion.
2. Give each paired attempt the same permitted files, runtime access, oracle queries, elapsed time, token budget, and tool-call budget.
3. Use fresh sessions or workspaces so the protected attempt cannot inherit a baseline solution. Vary ordering and record trial seeds.
4. Keep reference source, hidden tests, previous solutions, and ground-truth outputs out of the attack workspace. Score outside it.
5. Save failed attempts, compiler errors, timeouts, and inconclusive outcomes alongside successful reconstructions.

The first trial can be small enough to inspect manually. Repeated trials are needed before drawing a general conclusion. Report counts and uncertainty; do not describe one model failure as a universal defense.

Prevent benchmark overfitting

Develop on a declared training set. Freeze the protection approach before evaluating held-out programs, functions, build flags, or versions. If a holdout informs a redesign, it becomes development data and a fresh holdout is needed.

No protection rule should depend on the name PhotoCraft, reVC, Mattermost, an ImageLab symbol, or a known expected answer. Selection by a user-supplied annotation or generic policy is different: it is part of the supported interface and must work on unrelated software.

08 Evaluation records and scoring

A trial must be reproducible

Use a versioned trial manifest as an artifact. The following is a proposed record design, not additional fields already accepted by Core's strict models. Agree the schema with integration and link it to an authorized `attack_lab` job.

Record group	Required information
Build identity	Source revision, target profile, input and final hashes, tools, flags, dependencies, platform.
Protection	Backend version, configuration, seed, eligible coverage, transformed coverage, retention findings.
Authorization	Authorizer, reference, scope, dates, and permitted artifacts.
Attacker access	Available files, symbols, source-map policy, runtime/oracle access, network policy.
Attack configuration	Ghidra version and script, model/provider identity, prompt hash, sampling settings.
Budget	Time, tokens, tool calls, oracle queries, and actual consumption.
Scoring	Test-set identifier/hash, candidate hash, build result, passed checks, final outcome and reason.
Cost and evidence	Timings, memory where measured, artifact sizes, logs, trial order, limitations.

Proposed outcome interpretation: `bypassed` means the reconstruction meets the declared success criterion. `resisted` means a valid, completed attempt failed that criterion within the stated budget. `inconclusive` means the evidence cannot support either conclusion, such as contaminated inputs or an invalid comparison. `error` means the trial infrastructure failed. A timeout needs a reason: budget exhaustion and a broken worker are different events.

Metrics worth reporting

Report successful reconstructions as a count over completed valid trials, with all excluded and error trials shown separately. Record time and budget to success, partial test coverage, and failure modes. Add runtime, startup or frame-time cost, memory where applicable, and artifact-size change. State both matched-pipeline and normal-release baselines.

Do not compress all of these into an unexplained security score. The optimizer can later compare valid candidates under explicit cost limits. If none meets those limits, it should report that outcome rather than select a broken or unevaluated candidate.

Open source and model access

Public target source may already be known to a model. A source-withheld workspace does not establish that the model has never seen it. Disable repository retrieval when that is part of the trial, record allowed network access, and disclose possible prior knowledge. Use fresh private variants or other unseen authorized targets to complement public examples.

Nebius is the intended model-hosting route for the Lab issue. The model name, endpoint, settings, and GPU use must come from the actual run. No model deployment or NVIDIA acceleration is established by this documentation snapshot.

09 Development runbook

Establish the environment

The commands in this section use **Bash** on Linux. In Fish, run `bash` before these blocks and `exit` when finished. This avoids shell-specific command-substitution and heredoc errors. All paths are examples for a fresh local checkout; inspect an existing worktree before switching or installing anything.

Required tools are Git, Python 3.11 or newer, uv, CMake, Ninja, and matching LLVM/Clang 22.x development tools. Recorded versions were Python 3.14.7, LLVM/Clang 22.1.8, and CMake 4.4.3. Confirm installed tool paths rather than assuming `/usr/bin` points to the correct version. [S1, S6]

```
git clone https://github.com/Youssef-Boussabah/AEGIS.git
cd AEGIS
git switch feat/core-shield
git merge --ff-only origin/dev/integration
git rev-parse HEAD

uv venv .venv
uv pip install --python .venv/bin/python \
  -c core/requirements-dev.lock \
  -e './core[dev]' -e './platform/api[dev]' \
  -e './platform/cli[dev]'
.venv/bin/pytest

clang --version
opt --version
llvm-config --version
command -v clang opt llvm-config
```

The root is not an installable Python package. Install Core, API, and CLI together in the repository-local virtual environment. The default root pytest selection covers only those three package suites.

Build the real Shield plugin

```
cmake -S protectors/llvm -B /tmp/aegis-shield-build \
  -G Ninja -DLLVM_DIR="$(llvm-config --cmakedir)" \
  -DCMAKE_CXX_COMPILER=clang++ \
  -DCMAKE_BUILD_TYPE=Release
cmake --build /tmp/aegis-shield-build -j2
ctest --test-dir /tmp/aegis-shield-build \
  --output-on-failure
```

The plugin, `opt`, and Clang must use a compatible LLVM installation. A binary plugin from another LLVM build can fail to load. If a required toolchain is missing, record the test as not run and arrange the dependency through the project's contributor rules.

09 Register and test the adapter

Configure a fresh checkout

Run `.venv/bin/aegis init` once from the repository root. It creates `aegis.toml` and the local store, with no registered backend. If a configuration already exists, inspect it; do not force a replacement.

Add the following section in a text editor. Replace `/absolute/repo` with the path printed by `pwd`, and use the matching absolute `opt` path from the previous page. Keep the existing storage and attack-lab settings.

```
[adapters.llvm-shield]
command = [
  "/absolute/repo/.venv/bin/python",
  "/absolute/repo/protectors/llvm/adapter/aegis_llvm_adapter.py",
  "--opt", "/usr/bin/opt",
  "--plugin", "/tmp/aegis-shield-build/AegisShield.so"
]
tasks = ["protect"]
timeout_seconds = 300
env_passthrough = []
max_stdout_bytes = 65536
```

Do not add the section twice. `aegis inspect` checks whether the registered executable is available; it does not fully validate the plugin and LLVM configuration until a job runs. No attack-lab task is supported by this adapter. [S7]

Run a real fixture and the combined suite

```
.venv/bin/aegis inspect
clang -O1 -S -emit-llvm \
  protectors/llvm/tests/fixtures/annotated.c \
  -o /tmp/aegis-adapter-input.ll
.venv/bin/aegis protect /tmp/aegis-adapter-input.ll \
  --backend llvm-shield \
  --option 'seed="18446744073709551615"' \
  --option max-sites=16

.venv/bin/python -m pytest \
  core/tests platform/api/tests platform/cli/tests \
  protectors/llvm/adapter/tests \
  --shield-plugin /tmp/aegis-shield-build/AegisShield.so \
  --shield-opt "$(command -v opt)" \
  --shield-clang "$(command -v clang)" -q
```

The seed's inner double quotes matter: the CLI parses JSON scalars, while the adapter requires a decimal string. Use the returned job ID with `aegis report`. The 199-test record used an explicit combined invocation like this; `.venv/bin/pytest` alone is not equivalent.

09 Reproduce the application workflow

Generate the ImageLab baseline

This creates native and IR-baseline variants and prepares `kernels.input.ll`. Use a clean build directory for each configuration so cached options cannot silently add variants.

```
cmake -S demo/image_processor \
  -B /tmp/aegis-imagelab-baseline -G Ninja \
  -DCMAKE_CXX_COMPILER=clang++ \
  -DCMAKE_BUILD_TYPE=Release
cmake --build /tmp/aegis-imagelab-baseline -j2
ctest --test-dir /tmp/aegis-imagelab-baseline \
  --output-on-failure

.venv/bin/aegis protect \
  /tmp/aegis-imagelab-baseline/ir/kernels.input.ll \
  --backend llvm-shield \
  --option 'seed="18446744073709551615"' \
  --option max-sites=100
```

Rebuild from the returned artifact

The next block prompts for the actual job ID. It assumes the generated default `.aegis/` store; adjust the artifact path if your configuration uses a different root. Inspect the report before building.

```
read -r -p "Paste the successful ImageLab job ID: " AEGIS_JOB
.venv/bin/aegis report "$AEGIS_JOB"
.venv/bin/python -m json.tool \
  ".aegis/jobs/$AEGIS_JOB/outputs/shield-report.json"

cmake -S demo/image_processor \
  -B /tmp/aegis-imagelab-external -G Ninja \
  -DCMAKE_CXX_COMPILER=clang++ \
  -DCMAKE_BUILD_TYPE=Release \
  -DIMAGELAB_EXTERNAL_KERNELS_IR="$PWD/.aegis/jobs/$AEGIS_JOB/outputs/protected.ll"
cmake --build /tmp/aegis-imagelab-external -j2
ctest --test-dir /tmp/aegis-imagelab-external \
  --output-on-failure
```

Expected baseline counts from the documented configurations are 9 CTests without a protector, 11 with external IR, and 12 with the direct Shield plugin. Counts can change with repository revisions; compare named tests and build options, not only the total. [S8]

Save source commit, configuration, tool versions, report and artifact hashes, and CTest output. A successful run proves this chain works under those tests. Retention inspection and adversarial evaluation remain separate tasks.

10A Core platform and integration

Tracking: Issue #9. Branch: feat/core-shield. Role A covers this page and Shield development on the next page. Roles are independent of person assignments.

Role A owns Core, API, CLI, LLVM Shield and its adapter, ImageLab, root governance, docs and CI. This page covers platform and integration work; the next page covers Shield. Keep both compatible as roles B, C and D develop.

First implementation sequence

1. Reproduce the current package, plugin, adapter, and ImageLab tests from a clean clone. Record toolchain and exact commands.
2. Add CI jobs that exercise those actual suites. Pin the compatible LLVM toolchain, retain logs, and make missing required tools fail visibly.
3. Keep root guidance, both PDFs and module instructions aligned with roles A-D. Update them alongside implementation and contract changes.
4. Establish pull-request review, templates for test evidence, and agreed protection of `dev/integration`. Confirm the desired release branch separately; no new default branch is assumed here.
5. Agree only the interfaces needed by the first deliverables: experiment artifact schema, provenance linkage, and dashboard API gaps.

Deliverable and acceptance

A teammate should be able to clone, build, run the real fixture job, rebuild ImageLab from its output, and see the resulting report. CI must distinguish unit tests using fake adapters from integration tests using the real plugin. Documentation should reproduce the same configuration without machine-specific paths.

Acceptance includes explicit failures for unavailable tools, zero-transform jobs, malformed outputs, and unauthorized Lab requests. A green badge cannot substitute for an unexecuted required suite. Attach the release walkthrough and test results to the review.

Optimizer follows measurement

The later optimizer should enumerate a bounded set of supported policies, reject candidates that fail correctness or cost constraints, and compare the remaining measured results. Preserve seeds and artifact lineage. Do not optimize only for transformed-site count or an unexplained protection score.

Start with explicit user limits on runtime and size overhead. Retain candidates that offer different useful trade-offs rather than forcing one universal best setting. If evidence is too weak to choose, return the candidates and the uncertainty.

Dependencies and handoff. Shield supplies candidate artifacts and compiler information. Lab supplies reproducible evaluations. Mirage supplies a control and decoy variant. Signature supplies a verifiable manifest. The dashboard consumes the agreed APIs. Integration owns compatibility tests and the final combined run, not each module's internal algorithm.

10A Shield development

Tracking: Issue #9. Branch: feat/core-shield. This is the Shield part of role A.

Own `protectors/llvm/`: the plugin, its local schemas, adapter, tests, and benchmarks. Improve native-code protection under realistic compiler optimization while preserving the existing semantics and error behavior. The interface to Core remains the adapter protocol.

First implementation sequence

1. Reproduce the current fixture and ImageLab results. Capture both intermediate IR and final object or executable code.
2. Establish retention and overhead baselines at the intended release settings. Separate effects caused by the special IR route from effects caused by the transformation.
3. Compare a small number of transformation candidates with a written semantic argument. Select one that can be implemented and tested properly within the first delivery.
4. Implement it behind explicit configuration. Define eligibility, skip reasons, deterministic behavior, and limits on code growth.
5. Extend the report and adapter validation together if local schema changes are needed. Coordinate any shared protocol change with integration.

Deliverable and acceptance

Deliver one additional working transformation, its tests, reproducible measurements, and a description of its supported domain. Tests must cover selected and unselected functions, supported widths and types, wraparound and overflow conditions, control-flow interactions where relevant, and invalid configurations.

Run LLVM verification and independent runtime comparisons. Include ordinary optimization and final linking. Show what remains in the final artifact and report cases where protection disappears. Keep the no-transform failure behavior intact rather than counting unsupported inputs as successful protection.

For performance, compare against both the matching IR baseline and an ordinary optimized native build. Record hardware, compiler, flags, seed, warmup, sample count, input, and variability. A single measured workload does not establish a universal overhead figure.

Design constraints

Do not remove `nsw` or `nuw` flags blindly to create more eligible sites. Any change to eligibility must preserve LLVM's semantics. Treat pointer arguments, floating point, vectors, exceptions, concurrency, and undefined behavior as explicit design questions rather than extending claims implicitly.

Do not hard-code ImageLab functions or external application names. Generic annotations, configuration, and format checks are the selection mechanisms. Changes to generated instruction metadata and pass ordering must be reflected in repeat-run tests.

Dependencies and handoff. Send the Lab the final baseline/protected artifact pair, hashes, symbols policy, and exact build recipe. Send integration the revised local report schema and compatibility notes. Mirage should request shared LLVM hooks through this owner rather than editing the same files independently.

10B Adversarial AI Lab

Tracking: Issue #10. Branch: feat/lab-mirage. Role B owns attack_lab/ and protectors/mirage/. Deliver the Lab comparison before Mirage research.

Produce a repeatable comparison of behavioral reconstruction from unprotected and protected artifacts. Start with a small authorized ImageLab target. Ghidra supplies static analysis; the intended Nebius-hosted model proposes a reconstruction. An independent scorer decides whether the candidate meets the task, not the model's own narrative.

First implementation sequence

1. Define the task, allowed attacker access, success condition, and equal budgets. Agree a versioned trial manifest with integration.
2. Build deterministic artifact preparation and a Ghidra invocation with saved version and script identity. Keep hidden source and tests out of the attack workspace.
3. Add a provider interface for model calls. Record actual model identity, sampling settings, token usage, failures, and elapsed time without exposing credentials.
4. Compile and execute candidate code in an isolated worker. Separate build failure, runtime failure, test failure, budget exhaustion, and infrastructure error.
5. Return a real `attack_lab` result through the adapter contract, with the detailed trial report saved as an artifact. Run baseline/protected attempts in fresh sessions.

Deliverable and acceptance

Deliver one complete comparison that another engineer can rerun, including artifact hashes, request settings, sanitized transcripts, candidate code, hidden-test version, and scoring results. Include negative tests for missing or expired authorization, unavailable analysis tools, malformed model output, and worker failure.

Keep both existing authorization gates. A configuration switch is not enough, and an attestation without the operator switch is not enough. The test scope must identify the permitted artifacts. Do not default-enable attacks for an easier demo.

Before any security claim, show that baseline and protected tasks are comparable and that the scorer cannot leak hidden answers into retries. If the baseline task cannot be reconstructed either, investigate difficulty and budget before crediting the protection.

Isolation and provider boundaries

The Core adapter runner is not the execution sandbox. Write an isolation design before running model-generated code. It should define filesystem access, network restrictions, process and resource limits, teardown, and where provider tokens live. Infrastructure changes require the repository's architecture review process.

The coding agent chosen by a teammate and the reconstruction model used in experiments are separate choices. Engineers may use any agent; experiments need a pinned model configuration and comparable conditions.

Handoff. Give integration an example manifest, outcome mapping, schema, and real result. Give Shield and Mirage actionable failure analysis. Give the dashboard aggregated results with links to evidence, while preserving adapter-claim attribution.

10B Mirage research

Tracking: Issue #10. Branch: feat/lab-mirage. This is the Mirage part of role B. Own protectors/mirage/; coordinate shared LLVM changes with role A.

Test whether harmless decoys change reverse-engineering outcomes. A useful decoy must survive the build, preserve application behavior, and affect analysis or reconstruction. Additional functions alone do not demonstrate value.

First implementation sequence

1. Choose one technique with a narrow, explicit behavioral contract. Describe the intended analyst confusion and how it can be measured.
2. Implement deterministic generation from a recorded configuration and seed. Include a disabled mode and a matched no-decoy control.
3. Build both conditions through the real compiler and linker. Inspect the final artifact for retention; dead-code elimination may remove unused structures.
4. Run the same independent functional tests on both artifacts. Measure runtime and size costs under comparable build settings.
5. Give the Lab a paired experiment that can observe incorrect function selection, reconstruction success, or extra budget consumption.

Deliverable and acceptance

Deliver the transformation or generation tool, a reproducible sample, correctness tests, retention evidence, and an overhead report. The implementation must not add unintended file access, network calls, data collection, destructive behavior, or application-visible changes.

Explain what the attacker can observe and how success is scored. Retain unsuccessful results. Time lost to uploads or broken tools is not evidence of effective deception.

Integration design

Agree any LLVM changes, build flags, or new backend with the relevant owner. Shared changes in `protectors/llvm/` belong with Shield. Platform integration is complete only when a real job invokes the technique and records its artifacts.

Do not change annotation or allowlist behavior in ImageLab merely to produce a favorable score. Add target-specific setup to the test harness, keeping the generic decoy implementation independent of benchmark names.

Scope of the first research result

Record whether decoys remain after stripping and optimization, whether analysts choose them, and whether that changes reconstruction outcomes. Test symbols-present and stripped conditions separately if both matter to the proposed mechanism.

Mirage's first objective is behavioral deception through program structure. Instructions embedded in strings that ask an AI to stop analyzing do not establish reliable enforcement or protection. Keep those claims separate from measured decoy effects.

Handoff. Send Shield and integration the required build hooks and configuration limits. Send the Lab hashes, expected functional behavior, and the no-decoy control. Send the dashboard only measured retention, cost, and evaluation results.

10C Frontend & Product Experience

Tracking: Issue #11. Branch: feat/frontend. Role C owns platform/dashboard/ and presentation assets. Coordinate API and ImageLab code changes with role A.

Build a local interface around the real API so users can submit supported inputs and understand what changed, which checks ran, and what remains untested.

First implementation sequence

1. Reproduce a real CLI job and read its API models and report. Design the first screen around that actual information.
2. Implement a backend choice, supported-input guidance, policy options, job submission, status polling, and report presentation.
3. Add empty, pending, running, succeeded, failed, unauthorized, and unavailable-backend states. A pending report is not an application crash.
4. Coordinate missing API capabilities with integration. The snapshot exposes submission, status, report, and health; job listing and artifact downloads require additional work.
5. Write an ImageLab demonstration script that starts from a clean documented setup and ends with the protected application checks.

Deliverable and acceptance

Deliver a complete UI journey backed by an actual LLVM job. Show its ID, artifact hashes, configuration, outputs, failures, and evidence source. Transformation counts must come from the real report.

Label demo data used during development. Test key interactions and errors, then run the real end-to-end acceptance case. Demonstration data cannot become submission evidence.

Reporting and artifact access

The report should help a reader distinguish output integrity, functional testing, transformation retention, and attack evaluation. A success label refers to the job's contract; it is not a global security rating. Missing evaluations should appear as not run rather than zero attacks succeeded.

Do not expose arbitrary filesystem paths to create a download feature. Artifact retrieval needs authorization and path containment through an agreed API. Handle large base64 submissions within the server's limits and avoid caching sensitive uploads in browser storage without a deliberate requirement.

Demonstration sequence

Show an unprotected application result, submit its prepared IR, display the real job and report, rebuild using the returned IR, and run the correctness checks. Add a paired reconstruction result only once the Lab supports it. Add signing and decoys only when their real implementations pass acceptance.

Capture the exact commit and toolchain used for the recording. Keep a local replay path if a model service is unavailable, and label previously recorded trial evidence with its original conditions.

Handoff. Integration owns ImageLab's implementation and shared API edits. This workstream owns the user interface and presentation. Provide a setup guide, automated UI checks, the demo runbook, and a list of API dependencies with their owners.

10D Signature, Sentinel & Verification

Tracking: Issue #12. Branch: feat/signature-sentinel. Role D owns provenance/ and sentinel/. Start with offline signing; policy and watermark work follow.

Create verifiable build records and rights metadata. Signed provenance, watermarking, and policy interpretation need separate tests; none alone establishes reverse-engineering resistance.

First implementation sequence

1. Propose a versioned manifest covering artifact SHA-256, source/build reference, toolchain, configuration identity, and signing-key identifier.
2. Specify canonical bytes and a signature envelope. Use a maintained cryptographic library and a documented standard; do not design custom cryptography.
3. Implement offline verification against an explicit trusted public key. Keep signing keys outside the repository, logs, and ordinary artifact reports.
4. Add a versioned Sentinel policy that names the issuer, artifact linkage, intended permissions, and interpretation rules for cooperating tools.
5. Investigate a small recoverable watermark as a distinct optional experiment, including a detector and unmarked controls.

Deliverable and acceptance

Deliver manifest creation, verification, documentation, and negative tests. Altering an artifact or a covered manifest field must fail verification. Test a wrong key, malformed envelope, unsupported version, and truncated data. Define how absent or untrusted keys are reported; successful parsing is not trusted verification.

Sentinel should validate supported policy records and reject ambiguous or unsupported versions. The dashboard must distinguish an attached notice from a policy that an actual cooperating tool enforces. No claim of binding third-party agents follows from publishing JSON.

Watermark research

Specify embedding and extraction, supported transformations, payload capacity, and false-positive behavior. Test an unmarked artifact and a differently marked artifact as controls. Measure what remains after compilation, optimization, stripping, and other declared changes.

A manifest verifies its signer's statement, not ownership of all underlying intellectual property. A watermark's attribution strength depends on its design and measured false-positive rate. State these limits in the report.

Integration and lifecycle

Plan signing after the final bytes of the covered artifact are fixed. Later transformation changes its digest and requires a new record. Decide which intermediate artifacts need separate manifests, and preserve their linkage without introducing a circular signature dependency.

Integration owns new shared Core fields or workflow stages. Review any new task kind explicitly; the existing `protect` and `attack_lab` enums do not include signing.

Handoff. Provide a sample manifest and policy, public verification material, verified/tampered examples, and clear UI states. Never put private demonstration keys into a public submission.

11 Integration and release

Finish these before parallel feature work depends on them

dev/integration is the default integration branch. Both guides and the repository now use roles A-D and the same four feature branches. See CONTRIBUTING.md for checkout, synchronization and PR rules. CI and branch protection remain separate infrastructure work; documentation does not enable them. [S1, S10]

Required check	What a successful run must exercise
Python packages	Core, API, and CLI contracts and error paths.
LLVM plugin	Configure, compile, and the real CTest cases.
LLVM adapter	Explicit pytest suite with real plugin, opt, and clang paths.
ImageLab	Native/IR baseline, direct Shield, and AEGIS external-artifact path.
Changed modules	New behavior and negative cases in each feature's own suite.

Keep toolchain-dependent checks on suitable runners. Do not silently skip them and still declare the integration verified. Preserve logs and hashes needed to investigate a failure. Choose required status checks only after their names and behavior are stable.

Merge and handoff rules

Use focused feature branches from the agreed integration base. Work in separate worktrees where useful; the repository warns that all worktrees share one stash stack. Avoid rewriting another contributor's branch or integration history. Follow `AGENTS.md` for repository operations and test requirements.

A review should include the concrete behavior change, owned paths, affected interfaces, commands and results, and known limitations. Shared model, protocol, schema, storage, and security changes need coordinated review and documentation; significant architecture decisions need an ADR. The current rules require all affected checks, including the broader suite for root/documentation changes.

Deliver in dependency order

First make the existing build reproducible and establish ownership. Next, deliver Shield retention measurements and a small real Lab comparison. Then integrate independently tested Mirage and Signature/Sentinel work, with the dashboard presenting actual states. Add an optimizer only after correctness, cost, and attack measurements are trustworthy. Expand backend support one tested profile at a time.

The minimum credible demonstration is a real protection job, a rebuilt application, passing independent checks, and a transparent evidence report. Add a Lab comparison when implemented.

Decisions still to make

Select and pin the first external components; define Lab isolation and its result schema; choose a model deployment; decide signing format and trust management; agree API extensions and multi-stage artifact lineage; set cost budgets and release-branch policy. Person assignments are separate from this technical plan.

12 Agent working brief

Prepare the assignment

Any coding agent can use this brief. Fill in the fields and supply both documents plus repository access. The task owner must name allowed repository actions; this document is not blanket permission to commit, push, deploy, or change another module.

```
Workstream and issue:
Current branch and base commit:
Allowed paths:
First deliverable:
Acceptance checks:
Dependencies and owners:
Allowed repository actions:
```

Copy the following context and instructions

You are an engineer working on AEGIS. Its goal is software protection across desktop applications, websites and web apps, mobile apps, services, games, embedded systems, and authorized console builds. The product combines compatible protection backends with correctness checks, provenance, and controlled reverse-engineering evaluation. Platform support must be demonstrated for specific toolchains and artifacts.

Read AEGIS_USER_GUIDE for the team overview and AEGIS_TECHNICAL_DOCUMENTATION for the architecture and assignment. Read repository AGENTS.md, ARCHITECTURE.md, the relevant module README, and the issue. Inspect git status, current branch and commit, and the source involved. Do not assume this document matches a later revision. Report conflicts with concrete paths and explain which behavior the code and tests establish.

The documentation baseline at c995857 contains Python Core, FastAPI API, CLI, a local artifact store, a subprocess adapter protocol, an LLVM Shield plugin and adapter, and ImageLab. The plugin expands selected scalar integer additions. The adapter accepts textual LLVM 22 IR, returns protected.ll and shield-report.json, and rejects zero transformations. It does not compile a final application. ImageLab's external-IR hook provides the separate build and correctness stage.

Recorded results include 199 combined Python tests and an earlier ImageLab application milestone with 43 sites across three functions and 11 external CTests passing. These are historical records, not tests you have run. Shield v2, Mirage, signed provenance, watermarking, Sentinel, the real reconstruction Lab, adaptive optimizer, dashboard, and additional backends remain proposed until current code and tests establish their implementation.

Core owns shared validation, jobs, artifact hashes, reports, and adapter execution. API and CLI use Core; they do not import protectors. Backends communicate through aegis.adapter.v1. Keep compiler-specific behavior in its backend. Core models reject extra fields; a proposed field in an experiment manifest is not automatically valid in a Core model.

Continue with the implementation and evidence requirements on the next page. Use the filled assignment to determine scope.

12 Agent implementation and handoff

Before editing, summarize the relevant implementation, identify the smallest useful deliverable, list the files to change, and state how you will test it. Explain any dependency on another owner. Then carry out the authorized work. Request coordinated review when shared models, protocols, schemas, security controls, or module boundaries must change.

Preserve intended program behavior and public interfaces. Do not remove or weaken tests to obtain a pass. Include negative cases and independent expected outputs where feasible. Treat overflow, undefined behavior, pass ordering, and final linking as part of compiler work. Report unsupported formats and no-op transformations explicitly.

Preserve subprocess argument arrays, minimal environment, timeouts and process-group termination, path containment, input immutability, and orchestrator-side hashing. Keep both Lab authorization gates. Treat the present adapter runner as bounded execution, not a sandbox for malicious artifacts or generated programs. Keep secrets out of source, logs, fixtures, reports, and commits.

Keep evidence sources truthful. Core-observed facts remain orchestrator evidence. Adapter transformation counts and evaluation outcomes remain adapter claims unless independently checked through a documented mechanism. A changed hash, valid IR, or passing functional test does not prove resistance to reverse engineering.

For experiments, freeze tool and model identity, build flags, protection settings, seeds, budgets, attacker access, and scoring rules. Keep hidden tests and reference source away from the reconstruction worker. Record every trial, including errors. Compare the final protected artifact with both a matched pipeline baseline and an ordinary optimized build.

ImageLab is a controlled regression target. PhotoCraft, reVC, and Mattermost are requested external evaluation targets, not names to recognize in the protection engine. Do not implement benchmark-specific shortcuts or tune against hidden answers. Reserve holdouts and report coverage and unsupported profiles honestly.

Work on the assigned branch and paths. Do not overwrite unrelated changes, rewrite other branches, add unreviewed infrastructure, or push without the task's authorization. Keep build products and private artifact stores outside commits. Update the affected module documentation and ADRs when required by repository rules.

End with this handoff

1. State the implemented behavior and list the files changed.
2. Give the branch and commit, or say the work is uncommitted.
3. List exact test commands, results, warnings, and checks not run with reasons.
4. Identify artifact/report locations and hashes relevant to the result.
5. Explain remaining limitations and failed experiments without hiding them.
6. Name unresolved contract questions and the owner needed to resolve them.
7. Give the next concrete task and enough context to resume it.

Persist this handoff with the project's working records. A later agent should verify the current source and read the handoff before continuing. No single conversation is the complete project history.

13 Reference index

Sources used for this revision

Repository links below are pinned to the reviewed commit where appropriate. Live issues express work requests; they do not prove the work is implemented. External repositories were checked for target identity and environment, not built or benchmarked during this revision.

S1 Project and contributor instructions. README, AGENTS, and ARCHITECTURE.

S2 Core behavior. Core README and orchestrator.py.

S3 Models. models.py. **S4 Interfaces.** API README and CLI README.

S5 Adapter contract. adapter-protocol.md.

S6 LLVM implementation. Shield README and Shield.cpp.

S7 LLVM adapter. Adapter README.

S8 Application harness. ImageLab README and CMakeLists.txt.

S9 Test record. Commit c995857 records the 199-test and Shield run. The fixture CLI output and ImageLab milestone come from the supplied project conversation; original ImageLab artifacts still need archiving.

S10 Planning. Issues #9-#12 and docs/TEAM_WORKSTREAMS.md define roles A-D. Issue #7 tracks PhotoCraft, reVC and Mattermost. Issues #1-#6 are historical.

S11 External desktop target. PhotoCraft. **S12 External game target.** re-GTA, with the reVC branch. **S13 External web platform.** Mattermost.

Terms used in both documents

Artifact: a stored file identified by its digest. **Backend:** an implementation that performs a supported task. **Adapter:** the executable bridge between Core and a backend. **IR:** compiler intermediate representation. **Retention:** whether a transformation remains in the final built artifact. **Oracle:** a source of expected behavior used to score a candidate. **Holdout:** evaluation data not used to choose or tune the technique. **ADR:** a repository record of an architecture decision.